# Interview Coding Practice

8 beginner questions in TypeScript, with step-by-step answers

### How to use this guide

1. Read the question and the steps.
2. Cover the answer and write the code yourself in `practice.ts`.
3. Run it: `node wk1/InterviewCodeQA/practice.ts`
4. Compare with the expected output, then check the answer.
5. Say the "Say it in the interview" line out loud.
6. Repeat tomorrow without looking. Typing it 3 times beats reading it 20 times.

### The 8 questions

| # | Question | Key idea |
|---|---|---|
| 1 | Reverse a string | Loop backwards |
| 2 | Count each character | Object as a counter |
| 3 | Find duplicates | Remember what you've seen |
| 4 | Biggest and smallest | Keep the best so far |
| 5 | Palindrome | Reverse, then compare |
| 6 | Remove duplicates | Add only if not there |
| 7 | FizzBuzz | `%` remainder, check "both" first |
| 8 | Count vowels | `includes()` + counter |

### Interview tips

- **Write the steps as comments first**, then turn each one into code.
- **Think out loud.** Explaining your idea earns credit even if you get stuck.
- It's fine to say *"Let me think about this for a moment."*
- Test with a small example in your head before running it.

# 1 Reverse a string

Turn `"Playwright"` into `"thgirwyalP"`.

**Steps**

1. Make an empty string called `reversed`.
2. Loop from the LAST letter to the FIRST letter.
3. Add each letter to `reversed`.
4. Print `reversed`.

**Expected output**

```
thgirwyalP
```

✏️ Cover the answer below and try it yourself first.

## Answer

```
const word1 = "Playwright";
let reversed1 = "";                          // empty box to build the answer

for (let i = word1.length - 1; i >= 0; i--) { // start at last letter, go backwards
  reversed1 += word1[i];                     // glue this letter on the end
}
console.log(reversed1);                      // thgirwyalP
```

🗣️ **Say it in the interview**

*"I loop from the last letter to the first and add each one to a new string."*

**Remember:** `+=` on text glues letters together: `"Pl" + "a"` → `"Pla"`.

## 2 Count each character in a string

Count how many times each letter appears in `"hello"`.

### Steps

1. Make an empty object called `count`.
2. Loop through each letter.
3. If the letter is new, start at 1. If seen before, add 1.
4. Print `count`.

### Expected output

```
{ h: 1, e: 1, l: 2, o: 1 }
```

🖍️ Cover the answer below and try it yourself first.

### Answer

```
const word2 = "hello";
const count: Record<string, number> = {};    // letter -> how many times

for (const letter of word2) {
  count[letter] = (count[letter] || 0) + 1;  // new letter starts at 0, then +1
}
console.log(count);                          // { h: 1, e: 1, l: 2, o: 1 }
```

**🗣️ Say it in the interview**

*"I use an object as a counter. For each letter I add 1 to its count."*

**Remember:** `(count[letter] || 0)` means "use the old count, or 0 if there isn't one yet". `Record<string, number>` tells TypeScript the keys are text and the values are numbers.

# 3 Find duplicates in an array

Find the numbers that appear more than once in `[10, 20, 30, 20, 10, 40]`.

### Steps

1. Make an empty array called `seen`, and one called `duplicates`.
2. Loop through each number.
3. If it is already in `seen` AND not yet in `duplicates` → add it to `duplicates`.
4. Otherwise add it to `seen`.
5. Print `duplicates`.

### Expected output

```
[ 20, 10 ]
```

✏️ Cover the answer below and try it yourself first.

## Answer

```
const nums3 = [10, 20, 30, 20, 10, 40];
const seen: number[] = [];                    // numbers we have met before
const duplicates: number[] = [];              // numbers we met twice

for (const num of nums3) {
  if (seen.includes(num)) {                   // met it before?
    if (!duplicates.includes(num)) {          // and not already saved?
      duplicates.push(num);
    }
  } else {
    seen.push(num);                           // first time -> remember it
  }
}
console.log(duplicates);                      // [ 20, 10 ]
```

🗣️ **Say it in the interview**

*"I remember every number I see. If I see it again, it's a duplicate."*

**Remember:** `list.includes(x)` answers true/false: "is x in the list?". `!` means NOT.

# 4 Find the biggest and smallest number

Find the max and min of `[45, 3, 90, 12, 67]`.

### Steps

1. Start `max` and `min` as the FIRST number.
2. Loop through each number.
3. Bigger than `max`? It becomes the new max.
4. Smaller than `min`? It becomes the new min.
5. Print both.

### Expected output

```
Max: 90
Min: 3
```

🖍️ Cover the answer below and try it yourself first.

## Answer

```
const nums4 = [45, 3, 90, 12, 67];
let max = nums4[0];                          // guess: first number is biggest
let min = nums4[0];                          // guess: first number is smallest

for (const num of nums4) {
  if (num > max) {
    max = num;                               // found a bigger one
  }
  if (num < min) {
    min = num;                               // found a smaller one
  }
}
console.log("Max: " + max);                  // Max: 90
console.log("Min: " + min);                  // Min: 3
```

### 🗣️ Say it in the interview

*"I start with the first number, then replace it whenever I find a bigger/smaller one." (Shortcut you can mention: Math.max(...nums4) and Math.min(...nums4))*

**Remember:** Use `let` (not `const`) for values that change inside the loop.

# 5 Is it a palindrome?

A palindrome reads the same forwards and backwards, like `"madam"`.

### Steps

1. Reverse the word (just like Q1!).
2. If reversed `===` original → palindrome.
3. Print the result.

### Expected output

```
madam is a palindrome
hello is NOT a palindrome
```

✏️ Cover the answer below and try it yourself first.

## Answer

```
function isPalindrome(word: string): boolean {
  let reversed = "";
  for (let i = word.length - 1; i >= 0; i--) {  // same as A1
    reversed += word[i];
  }
  return reversed === word;                     // same forwards and backwards?
}

const word5a = "madam";
const word5b = "hello";

if (isPalindrome(word5a)) {
  console.log(word5a + " is a palindrome");
} else {
  console.log(word5a + " is NOT a palindrome");
}
if (isPalindrome(word5b)) {
  console.log(word5b + " is a palindrome");
} else {
  console.log(word5b + " is NOT a palindrome");
}
```

### 🗣️ Say it in the interview

*"I reverse the word and check if it matches the original."*

**Remember:** A `function` lets you reuse the same steps for many words. `===` checks "exactly equal".

# 6 Remove duplicates from an array

Keep each number only once from `[1, 2, 2, 3, 1, 4, 3]`.

**Steps**

1. Make an empty array called `unique`.
2. Loop through each number.
3. If `unique` does NOT include it yet → push it in.
4. Print `unique`.

**Expected output**

```
[ 1, 2, 3, 4 ]
```

✏️ Cover the answer below and try it yourself first.

## Answer

```
const nums6 = [1, 2, 2, 3, 1, 4, 3];
const unique: number[] = [];

for (const num of nums6) {
  if (!unique.includes(num)) {              // not in the list yet?
    unique.push(num);                       // then add it
  }
}
console.log(unique);                        // [ 1, 2, 3, 4 ]
```

🗣️ **Say it in the interview**

*"I only add a number to the new list if it isn't there already." (Shortcut you can mention: [...new Set(nums6)])*

**Remember:** `push()` adds an item to the end of an array.

## 7 FizzBuzz

Print numbers 1 to 15, but: divisible by 3 and 5 → **FizzBuzz**; by 3 only → **Fizz**; by 5 only → **Buzz**; otherwise the number.

### Steps

1. Loop `i` from 1 to 15.
2. Check "3 AND 5" FIRST.
3. Then check 3, then 5.
4. Otherwise print `i`.

### Expected output

```
1  2  Fizz  4  Buzz  Fizz  7  8
Fizz  Buzz  11  Fizz  13  14
FizzBuzz
(each on its own line)
```

✏️ Cover the answer below and try it yourself first.

### Answer

```
for (let i = 1; i <= 15; i++) {
  if (i % 3 === 0 && i % 5 === 0) {          // check BOTH first!
    console.log("FizzBuzz");
  } else if (i % 3 === 0) {
    console.log("Fizz");
  } else if (i % 5 === 0) {
    console.log("Buzz");
  } else {
    console.log(i);
  }
}
```

🗣️ **Say it in the interview**

*"% gives the remainder. If the remainder is 0, it divides evenly. I check 'both' first, otherwise 15 would just print Fizz."*

**Remember:** `%` gives the remainder. `i % 3 === 0` means "i divides by 3 with nothing left over".

# 8 Count the vowels

Count the vowels (a, e, i, o, u) in `"Automation"`.

### Steps

1. Make a string of vowels: `"aeiou"`.
2. Make a counter starting at 0.
3. Loop through each letter (use `.toLowerCase()` so "A" counts too).
4. If `vowels.includes(letter)` → add 1.
5. Print the counter.

### Expected output

```
Vowels: 6
```

✏️ Cover the answer below and try it yourself first.

## Answer

```
const word8 = "Automation";
const vowels = "aeiou";
let vowelCount = 0;

for (const letter of word8.toLowerCase()) {  // lower case so "A" counts
  if (vowels.includes(letter)) {
    vowelCount++;                            // same as vowelCount = vowelCount + 1
  }
}
console.log("Vowels: " + vowelCount);        // Vowels: 6
```

### 🗣️ Say it in the interview

*"I go through each letter and add 1 whenever it's one of a, e, i, o, u."*

**Remember:** `count++` is short for `count = count + 1`.

## Cheat sheet: the building blocks

Almost every answer above is made from these few pieces.

| Code | What it means |
|---|---|
| `for (let i = 0; i < list.length; i++)` | Loop forwards, using a position number `i` |
| `for (let i = s.length - 1; i >= 0; i--)` | Loop backwards, from the last item to the first |
| `for (const item of list)` | Loop through each item directly (no number needed) |
| `text += letter` | Glue a letter onto the end of `text` |
| `list.push(x)` | Add `x` to the end of an array |
| `list.includes(x)` | true / false: is `x` in the list (or string)? |
| `count[key] = (count[key] \|\| 0) + 1` | Add 1 to a counter, starting at 0 if new |
| `a % b === 0` | `a` divides by `b` with nothing left over |
| `a === b` | Exactly equal |
| `!something` | NOT (flips true ↔ false) |
| `&&` `\|\|` | AND / OR |
| `x++` | Same as `x = x + 1` |
| `let` vs `const` | `let` can change later; `const` can't be reassigned |
| `Record<string, number>` | TypeScript type: object with text keys and number values |

**Bonus shortcuts to mention:** `Math.max(...nums)`, `Math.min(...nums)`, `[...new Set(nums)]`, `word.split("").reverse().join("")`. Learn the loop version first, because interviewers often ask for it.